universidade de aveiro
deti
departamento de eletrónica,
telecomunicações e informática

Distributed Systems
Architectures

Eurico Pedrosa <efp@ua.pt>

2^{nd} semester - 2025′26

Architectures

Architectures

- **Logical vs. Physical Organization**: Distinction between software architecture (logical component interaction) and system architecture (physical placement on machines).
- **Middleware & Transparency**: Use of a middleware layer to achieve distribution transparency, masking network complexity from the end-user.
  - e.g. Remote Procedure Calls (RPC).
- **Centralized Architectures**: Traditional client-server models where a central node manages functionality and responds to remote client requests.
  - e.g. Web servers or centralized database systems.
- **Decentralized & Hybrid Models**: Deployment of peer-to-peer (P2P) networks or hybrid architectures to balance scalability and control.
  - e.g. BitTorrent or Content Delivery Networks (CDNs).

Architectures Styles

An architectural style is a formal organization of a distributed system described in terms of its structural elements and their interactions. It is defined by four primary factors:

1. **Components**: Modular units of software.
2. **Connectors**: Mechanisms that mediate communication between components.
3. **Data**: The information exchanged between components.
4. **Configuration**: How these elements are jointly organized into a cohesive system.

Architectures Styles: Components

A component is defined as a modular unit with well-defined required and provided interfaces.

- **Encapsulation**: Components are replaceable within their environment without affecting the rest of the system, provided their interfaces remain untouched.
- **Maintenance**: High-performance systems often require component replacement or updates while the system continues to operate.
- **Continuity**: To avoid downtime, updates are often executed on a specific part of the system (e.g., a server) which then switches to the refreshed components.

Architectures Styles: Connectors

Connectors are mechanisms that enable the flow of control and data between components. Examples include:

- **Remote Procedure Calls (RPC)**: Facilities for invoking functions on remote machines.
- **Message Passing**: Systems designed for exchanging discrete messages.
- **Streaming Data**: Facilities for the continuous transfer of data.

Architectures Styles: Data

Data refers specifically to the information that is passed between components.

- **Interaction**: Data is the “payload” carried by connectors (like RPC or message passing).
- **Style Impact**: The nature of the data often dictates the style. For example, in a Shared Data Space, the style is defined by how data (tuples) are stored and retrieved associatively, rather than how components call each other.

Architectures Styles: Configuration

Configuration is the “blueprint” or “topology” of the system. It describes how components and connectors are jointly organized into a cohesive whole.

- **System Structure**: It defines the relationships and boundaries between parts, for instance, whether they are organized in a hierarchy (Layered) or as independent peers (Symmetric/ P2P).
- **Dynamic Changes**: In modern distributed systems, the configuration may change as components join or leave (e.g., in a peer-to-peer system), but the style provides the rules for how that configuration is allowed to evolve.

Primary Architectures Styles

Three major **Architectural Styles** are commonly identified

- **Layered**: Organized in a hierarchy where higher layers call upon lower layers
  - e.g. Communication-protocol stacks and the logical layering of applications into User Interface, Processing, and Data levels.
- **Service-Oriented (SOA)**: Independent services that communicate with each other.
  - This includes Object-based styles and Microservices
  - RESTful: A resource-based architecture popular for the Web
- **Publish-Subscribe**: Interact through an intermediary based on event notifications
  - e.g. “mailbox” coordination, event-based coordination, and shared data spaces

Architectures Styles: Hybrid Organizations

It is important to note that most real-world distributed systems do not adhere strictly to a single style.

- **Combination of Styles**: Designers frequently combine multiple styles within a single system.
- **Universal Layering**: Subdividing a system into logical layers is a universal principle often integrated with other styles, such as Service-Oriented Architectures.

Layered Architectures

Layered Architectures

The basic idea for the layered architectural style is to organize components in a hierarchical fashion.

- **Layer Organization**: Components are organized into layers where a component at layer L_j can make a call to a component at a lower-level layer L_i (where $i < j$)
- **Service Request**: A higher layer generally expects a response after making a “downcall” to a lower layer.
- **Universal Principle**: Subdividing a system into logical layers is a universal principle often combined with other architectural styles, such as Service-Oriented Architectures.

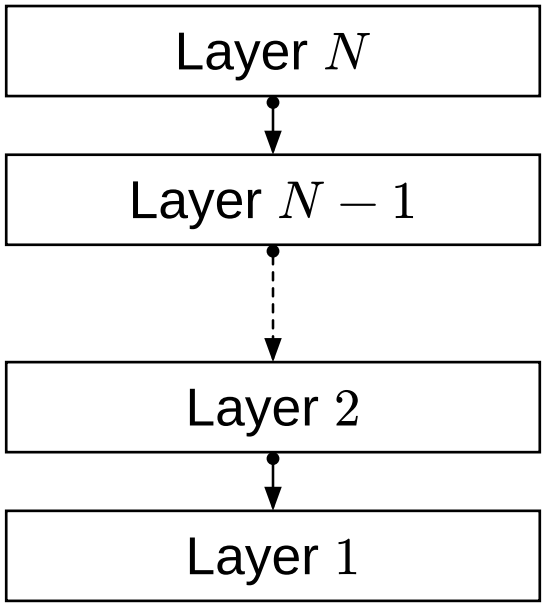
Layered Architectures: Interaction Types

Communication between layers typically follows one of three patterns:

1. **Pure Layered Organization**
   A standard organization where components only make downcalls to the immediate next lower layer.

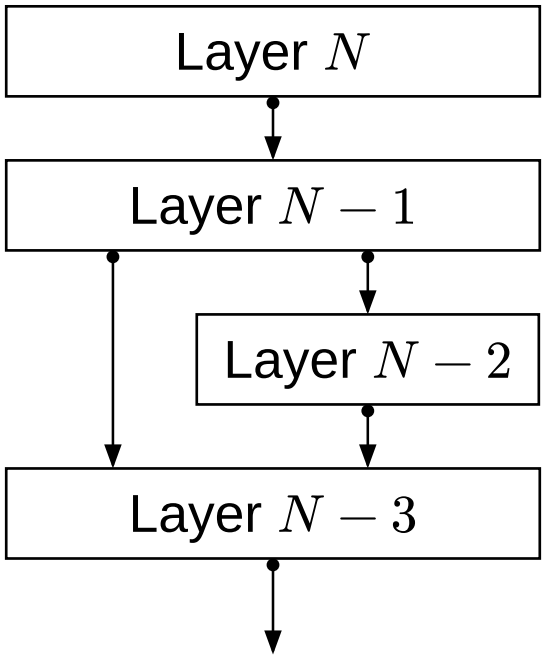
Request/Response

downcall



Layered Architectures: Interaction Types

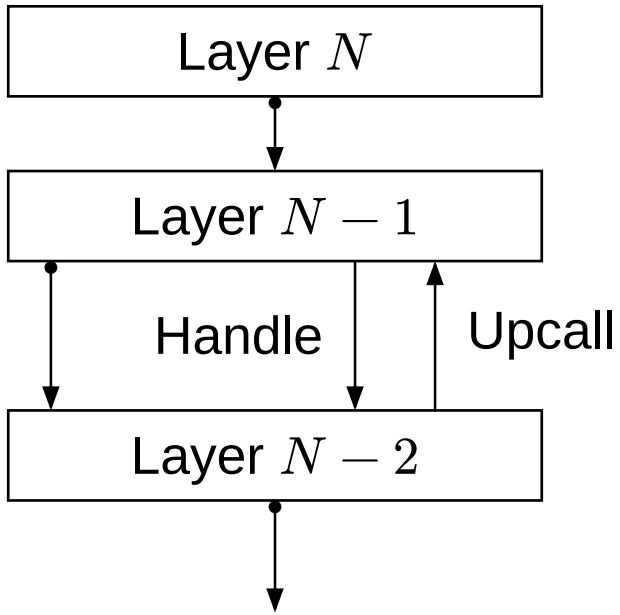
2. **Mixed Layered Organization**
   An application or component may make downcalls to any lower-level layer
   - e.g., an application using both a math library and an OS library

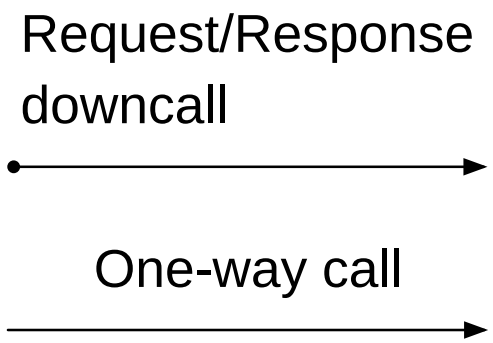


Request/Response downcall

Layered Architectures: Interaction Types

3. **Upcalls**
   In certain cases, a lower layer may signal a higher layer
   - e.g., an operating system might use a “Handle” to notify it of a specific event.





Layered Architectures: Communication Protocols

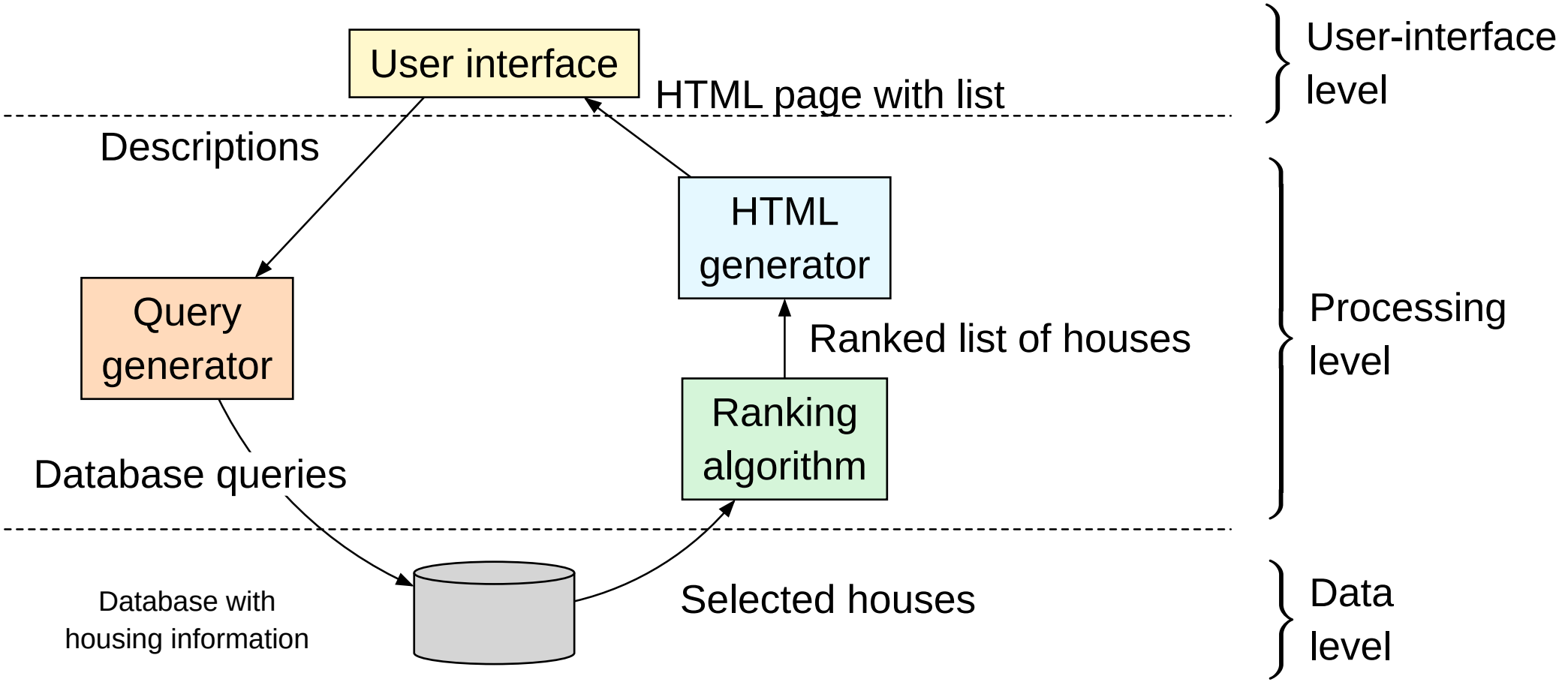
Layered architectures are most famously applied in communication-protocol stacks.

- **Service**: Each layer implements specific communication services for the layer above
  - e.g., reliable data transfer
- **Interface**: Specifies the functions that can be called, completely hiding the actual implementation of the service.
- **Protocol**: Describes the rules that peer entities follow to exchange information
  - e.g., a client and a server at the same layer

Layered Architectures: Application Layering

Most distributed applications can be logically divided into three distinct levels:

1. **Application-interface level**: Contains the parts of the application that handle interaction with users or external applications
   - e.g., a Web browser interface
2. **Processing level**: Contains the core functionality or “business logic” of the application
3. **Data level**: Manages the persistent data required by the system, typically stored in a database or file system.


User interface
HTML page with list
User-interface level
Descriptions
HTML generator
Query generator
Ranked list of houses
Processing level
Ranking algorithm
Database queries
Database with housing information
Selected houses
Data level

Layered Architectures: Drawbacks and Trade-offs

While highly structured, layered architectures have specific disadvantages:

- **Dependency Issues**: There is often a strong dependency between different layers, which can hinder modularity if interfaces are not stable.
- **Performance Overhead**: In some cases, the strict passing of data through every layer can introduce latency compared to more direct interaction styles.

Service-Oriented Architectures (SOA)

Service-Oriented Architectures (SOA)

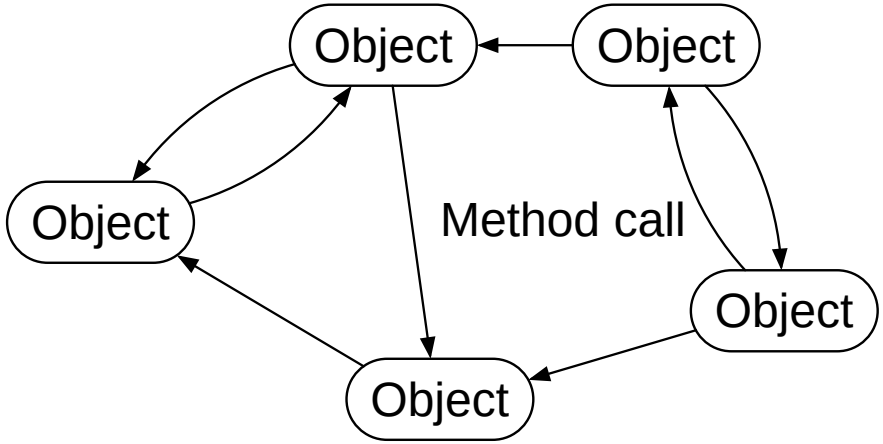
Service-oriented architectures (SOA) represent a shift toward a more loose organization of separate, independent entities compared to the strict dependencies found in layered styles.

- **Independence**: Each entity encapsulates a service that is typically executed as a separate process or thread.
- **Loose Coupling**: By separating services into independent units, the system avoids the instability caused by direct dependencies on specific internal component implementations.
- **Encapsulation**: The primary goal is to ensure that a service truly represents a separate, modular function.

SOA: Object-Based

The object-based approach provides a natural way to encapsulate data and operations.

- **State and Methods**: Each object encapsulates its data (state) and the operations that can be performed on that data (methods) into a single entity.
- **Interface Separation**: The interface offered by an object conceals implementation details, allowing it to be considered independent of its environment.



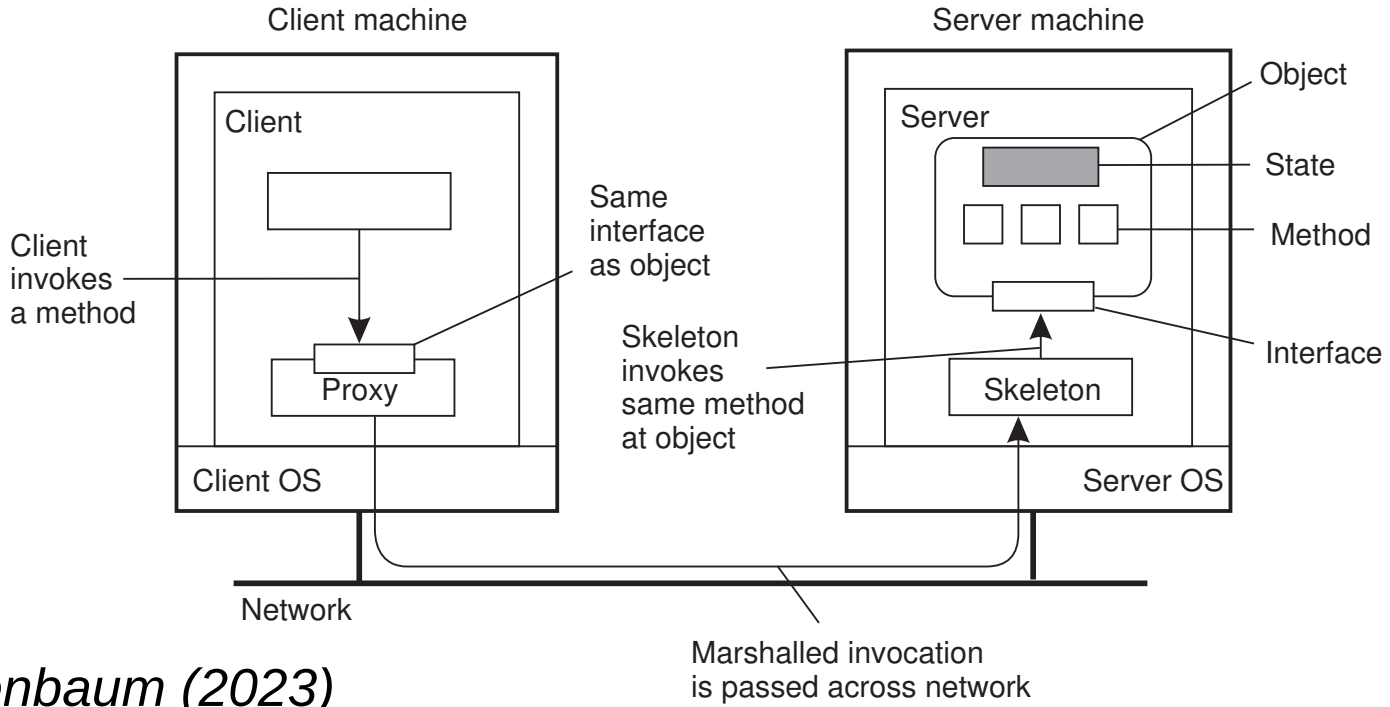
- **Replaceability**: As long as the interface remains untouched, an object can be replaced by another implementation without affecting the rest of the system.

SOA: Distributed and remote Objects

In distributed systems, the separation between interfaces and implementations allows for the creation of distributed or remote objects.

- **Physical Organization**: A characteristic feature is that the object's state typically resides on a single server machine, while its interfaces are made available on other machines
- **Client-Side Proxy**: When a client binds to a distributed object, a proxy (similar to a client stub in RPC) is loaded into its address space to pack method invocations into messages
- **Server-Side Skeleton**: On the server machine, a skeleton (or server stub) unpacks incoming requests and invokes the actual method at the object's interface

SOA: Distributed and remote Objects



Steen and Tanenbaum (2023)

SOA: Microservices

Microservices are a modern evolution of service-oriented design, drawing inspiration from the Unix philosophy of composing small, independent programs.

- **Process Isolation**: Each microservice runs as a separate, independent networked process.
- **Modularization**: Modularization is the key design principle, ensuring that each service can operate and be deployed independently.
- **Orchestration**: A significant challenge in microservice architectures is determining where to place and how to coordinate these services across different layers
  - e.g., edge vs. cloud

SOA: Coarse-Graine Service Composition

Distributed applications in an SOA are often constructed by composing multiple, potentially large-scale services.

- **Administrative Boundaries**: Composition often involves services owned by different organizations
  - e.g., a Web shop selling e-books while using a third-party cloud provider for storage
- **Integration**: The development process involves ensuring these diverse services operate in harmony, which is analogous to enterprise application integration.

SOA: Resource-Based Architectures (REST)

As service composition became more common, a resource-oriented alternative to traditional specific interfaces emerged, known as Representational State Transfer (REST).

- **Resource Identification**
  All resources are identified through a single, uniform naming scheme (typically URIs).
- **Uniform Interface**
  All services offer the same four generic operations to manipulate resources.

Operation	Description
PUT	Modify a resource by transferring a new state.
POST	Create a new resource.
GET	Retrieve the state of a resource in some representation.
DELETE	Delete a resource.

SOA: Characteristics of RESTful Architectures

RESTful designs emphasize simplicity and scalability through specific properties:

- **Self-Describing Messages**: Every message sent to or from a service contains all the information necessary to process it.
- **Statelessness**: The service component “forgets” everything about the caller after executing an operation, simplifying server recovery and scaling.
- **Access Transparency**: The focus on generic operations (GET/PUT) rather than specific API calls simplifies the interaction between different components.

SOA: Challenges

- **Microservice Complexity**: Determining where to place and how to coordinate numerous services across different layers is a major design challenge
  - e.g., edge vs. cloud
- **Process Isolation Management**: While running each service as an independent networked process improves modularization, it significantly increases management overhead
- **Administrative Boundaries**: Composing services owned by different organizations requires complex integration efforts
  - e.g., third-party cloud storage for a Web shop
- **Enterprise Integration**: Ensuring that diverse services operate in harmony is often compared to a “nightmare” of application integration.

SOA: Challenges

- **Semantic Gaps**: While IDLs specify the syntax (names and types), the actual semantics (what the service does) are often documented informally in natural language, leading to potential implementation mismatches.
- **Completeness vs. Neutrality**: It is difficult to create interface specifications that are complete enough for implementation without prescribing specific implementation details
- **Remote Interaction**: Relying on remote procedure calls (RPC) or remote method invocations (RMI) introduces latency and requires handling potential network failures that do not exist in local executions.

Publish-Subscribe Architectures

Publish-Subscribe Architectures

As distributed systems grow, minimizing dependencies between processes becomes critical. Publish-subscribe architectures achieve this by separating processing from coordination.

Core Concept: Decoupling

- **Loose Coupling**: The architecture aims for a strong separation where processes operate autonomously.
- **Coordination as Glue**: Coordination acts as the “glue” that binds the activities of independent processes into a functional whole.

Publish-Subscribe Architectures: Types of Decoupling

Coordination models are classified along two primary dimensions: temporal and referential coupling.

- **Referential Decoupling**: Processes do not need to know each other's identities or addresses. A publisher simply provides a notification, and the system handles delivery to interested subscribers.
- **Temporal Decoupling**: Communicating processes do not need to be active at the same time. In models like shared data spaces, a message (tuple) can be stored by the middle-ware until a subscriber is ready to retrieve it.

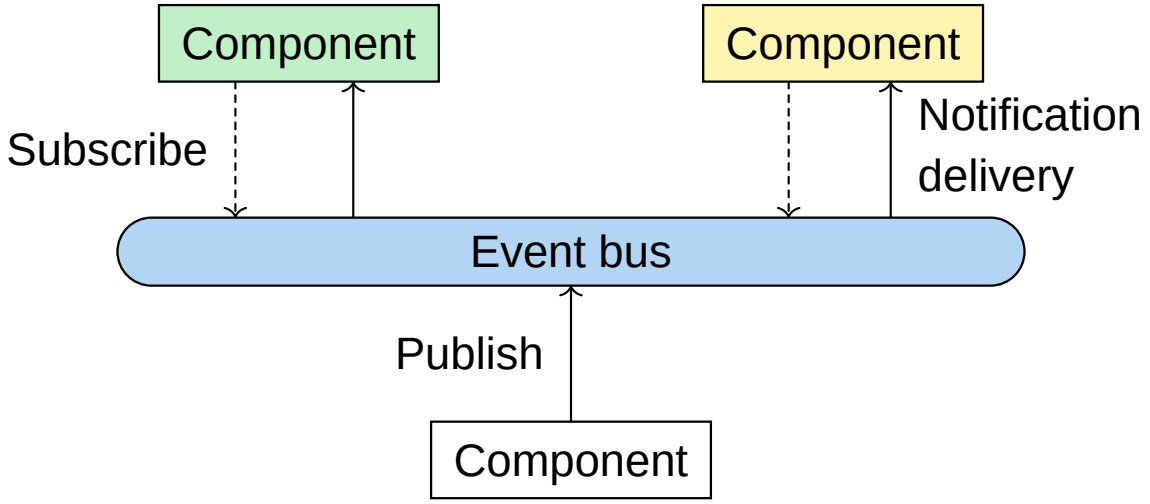
Publish-Subscribe Architectures: Coordination Matrix

	Temporally Coupled	**Temporally Decoupled**
Referentially Coupled	Direct Coordination (e.g., cell phone call)	Mailbox Coordination (e.g., email)
Referentially Decoupled	Event-based Coordination	Shared Data Space

Publish-Subscribe Architectures: Event-Based vs. Shared Data Space

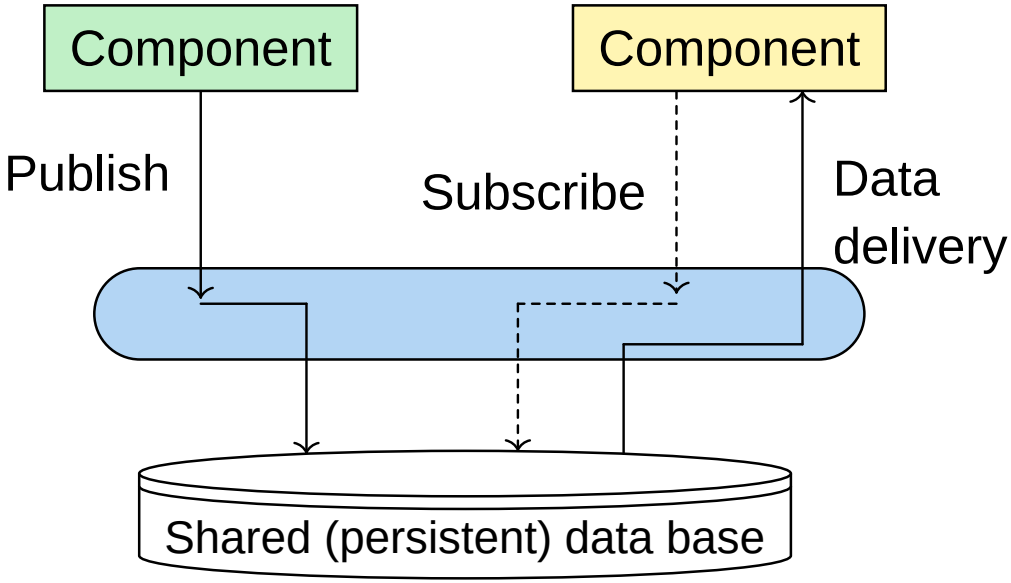
The publish-subscribe style is generally implemented through two main approaches:

- **Event-Based**: Components interact via an "event bus." A publisher sends a notification, and the middleware delivers it directly to subscribers who are currently active.



Publish-Subscribe Architectures: Event-Based vs. Shared Data Space

- **Shared Data Space**: Processes communicate through “tuples” (structured data records) stored in a persistent space. Processes can search for and retrieve tuples associatively based on their content.



Publish-Subscribe Architectures: Subscription Filtering Mechanisms

Matching subscriptions to published notifications is a central challenge, typically handled in two ways:

- **Topic-based**: Notifications are tagged with keywords (topics), subscribers indicate interest in specific subjects
  - e.g., “sd.deti.ua.pt”
- **Content-based**: Subscriptions are more expressive, using predicates over attribute values
  - e.g., “notify if temperature > 30°C”
  - This offers more flexibility but is harder to implement at scale.

Publish-Subscribe Architectures: Data Delivery Scenarios

When a match is found between a publication and a subscription, the system follows one of two patterns:

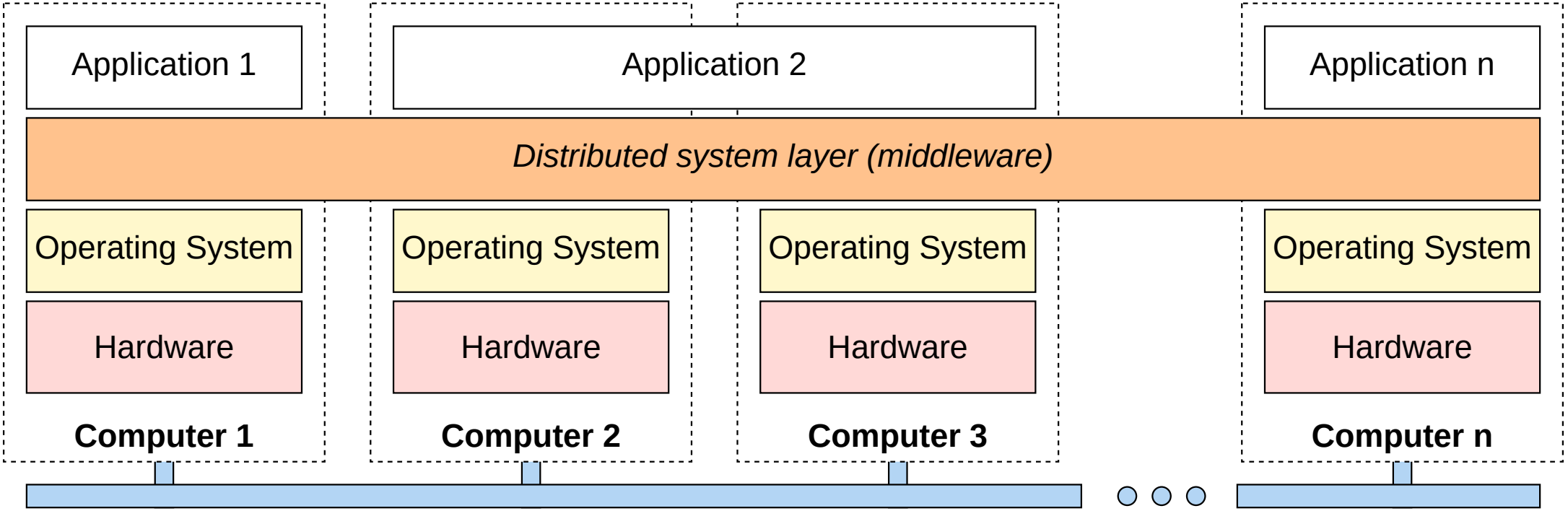
1. **Direct Delivery**: The middleware forwards the notification and the associated data directly to the subscribers.
2. **Notification only**: The middleware sends a notification, and the subscriber must explicitly perform a “read” operation to pull the data from the system.

Publish-Subscribe Architectures: Challenges

- **Scalability**: While the decoupling offers potential for large-scale systems, implementing efficient matching across many servers is complex, especially for content-based systems.
- **Event Composition**: Advanced systems may require “complex event processing,” where a subscriber is notified only if a specific sequence or combination of events occurs
  - e.g., “room is empty AND door is unlocked”

Middleware and Distributed Systems

Middleware and Distributed Systems



Middleware and Distributed Systems

- **Middleware as an Abstraction Layer**: Middleware is a specialized software layer situated between applications and local operating systems, designed to mask the heterogeneity of underlying hardware, platforms, and network protocols.
- **Achieving Distribution Transparency**: The primary purpose of middleware is to provide distribution transparency, ensuring that a collection of independent computers appears to users and applications as a single, coherent system.
- **Internal Organization via Architectural Styles**: Middleware is often logically organized according to specific architectural styles to manage the complexity of communication and coordination between distributed components.
  - e.g., layered, object-based, or publish-subscribe

Middleware Organization

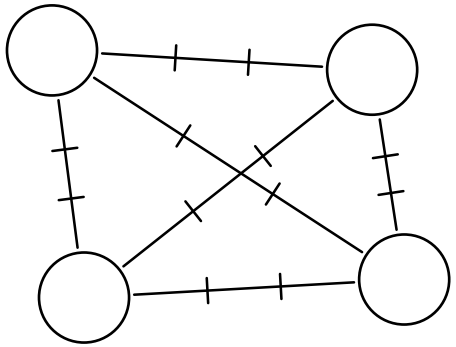
Middleware is a specialized software layer situated between applications and local operating systems.

- **Abstraction**: It masks the heterogeneity of the underlying hardware, operating systems, and network protocols
- **Transparency**: Its primary goal is to provide distribution transparency, making a collection of computers appear as a single coherent system
- **Independence**: It provides application-independent services, such as naming, authentication, and distributed transactions

Middleware Organization: Wrappers (Adapters)

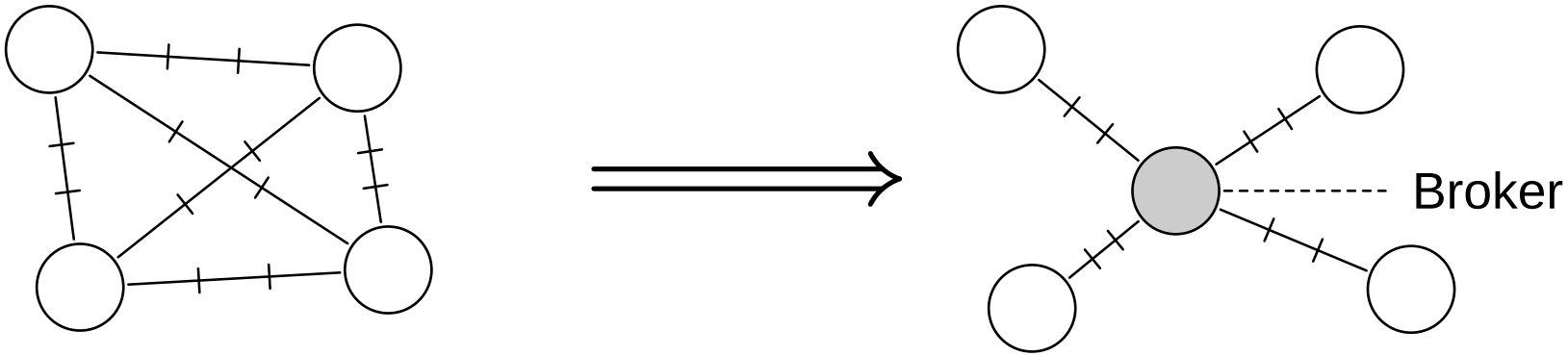
A common problem in middleware is integrating “legacy” components that were not designed for the specific middleware

- **Definition**: A wrapper is a component that provides a middleware-compliant interface to an underlying legacy application or object
- **Function**: It transforms the generic middleware requests into specific method calls or internal operations understood by the legacy component



Middleware Organization: Wrappers (Adapters)

- **Scalability**: While effective, creating a separate wrapper for every client-server combination can lead to a “wrapper explosion” if not managed through a message broker.

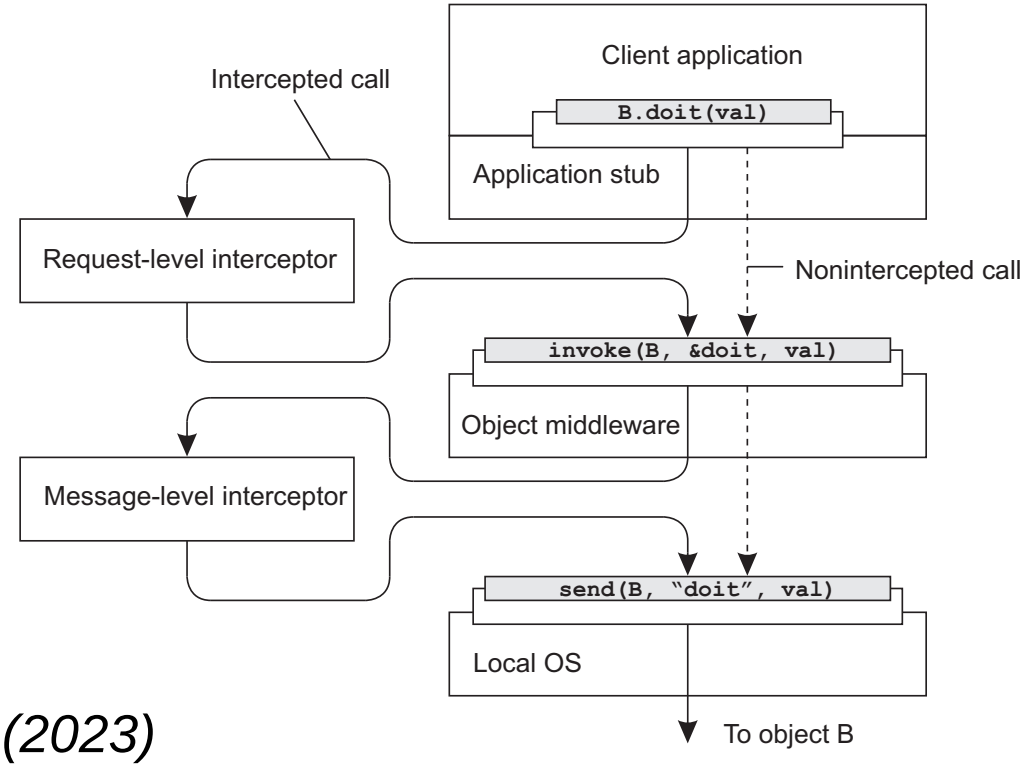


Middleware Organization: Interceptors

Interceptors allow for the modification of a middleware's behavior without altering its core implementation.

- **Mechanism**: An interceptor is a software construct that "breaks" the normal flow of control, allowing a developer to inject code into a request or response path.
- **Types**:
  - **Request-level Interceptors**: Invoked for every incoming or outgoing request
    - e.g., to handle replication or logging
  - **Message-level Interceptors**: Invoked for every incoming or outgoing message at the transport level
    - e.g., for encryption

Middleware Organization: Interceptors



Steen and Tanenbaum (2023)

Modifiable and Adaptive Middleware

Modern distributed systems require the ability to be updated or reconfigured dynamically.

- **Dynamic Loading**: Modern middleware platforms support adding or replacing components (like services or objects) while the system continues to operate
- **Component Frameworks**: Systems are often organized as a minimal “kernel” with pluggable components that provide specific policies for communication or fault tolerance
- **Self-Configuration**: Systems may observe their own behavior and dynamically adjust parameters to optimize performance
  - e.g., timeout values

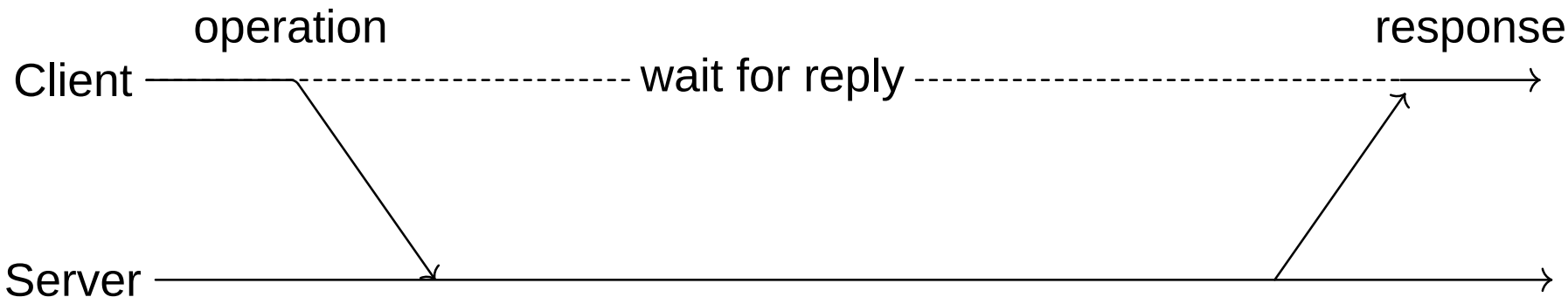
Layered-system Architectures

Layered-system Architectures

- **Physical Mapping of Logical Layers**
  System architectures represent the physical instantiation of a distributed system, defining how logical software components are mapped onto actual machines and networked nodes
  - e.g., the user interface, processing logic, and data management
- The **client-server model** is the most traditional way to physically distribute a system. It is based on the distinction between processes that request services (clients) and those that provide them (servers).

Simple Client-Server Architectures

- **Request-Response Cycle**: Communication is typically synchronous, where a client sends a request and blocks until the server sends a response.
- **Roles**:
  - **Client**: Initiates the interaction.
  - **Server**: Waits for incoming requests and processes them.

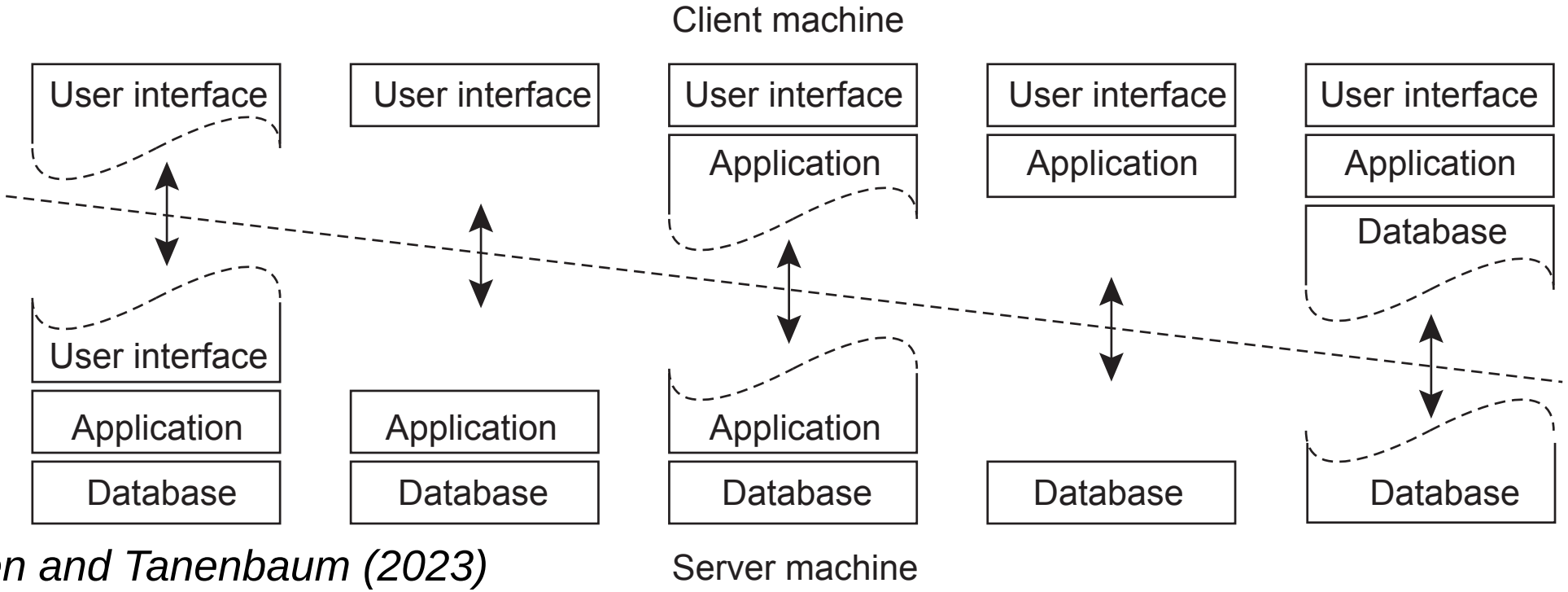


Simple Client-Server Architectures: Physical Mapping

In a simple client-server architecture, the system is physically split into two types of machines. The design focuses on how to distribute the three logical levels across these machines:

1. **User-interface level**: Always resides on the client side (part of)
2. **Data level**: Typically resides on the server side (database).
3. **Processing level (Application)**: Can be split or moved between the client and the server.

Simple Client-Server Architectures: Physical Mapping



Steen and Tanenbaum (2023)

Simple Client-Server Architectures: 2-Tier Variants

Depending on where the “business logic” (processing level) is placed, we distinguish several 2-tier configurations:

- **Thin Client**: The client machine only handles the user interface. All processing and data management occur on the server.
- **Fat Client**: The client machine handles both the user interface and the core processing logic. The server is reduced to a “data server” (e.g., a simple database).

Type	Client Responsibility	Server Responsibility
Thin Client	Display and UI interaction.	Business logic and Data.
Fat Client	UI and Processing logic.	Data storage and retrieval.

Simple Client-Server Architectures: 2-Tier Variants

Advantages and Trade-offs

- **Simplicity**: Easy to implement for small-scale applications and departmental systems
- **Performance**: Fat clients can reduce server load by performing computations locally
- **Maintenance**: Thin clients are easier to update, app logic is centralized on the server
- **Scalability Bottleneck**: The server often becomes a bottleneck as the number of clients increases, leading to the need for 3-tier or multi-tier organizations.

- **Example:** A Modern Web Application
  - **Client (Web Browser)**: Provides the interface to read and compose emails (UI)
  - **Server**: Manages the authentication, message processing, and storage in a database (Processing and Data levels)

Multitiered Architectures

Multitiered architectures arise from the need to physically distribute the three logical level (e.g., user-interface, processing, and data) across more than two machines.

- **Evolution of Client-Server**: While 2-tier models are simple, they often fail to scale when the processing or data levels become bottlenecks
- **Separation of Concerns**: By introducing more tiers, each physical machine can be optimized for a specific logical task
  - e.g., a dedicated application server

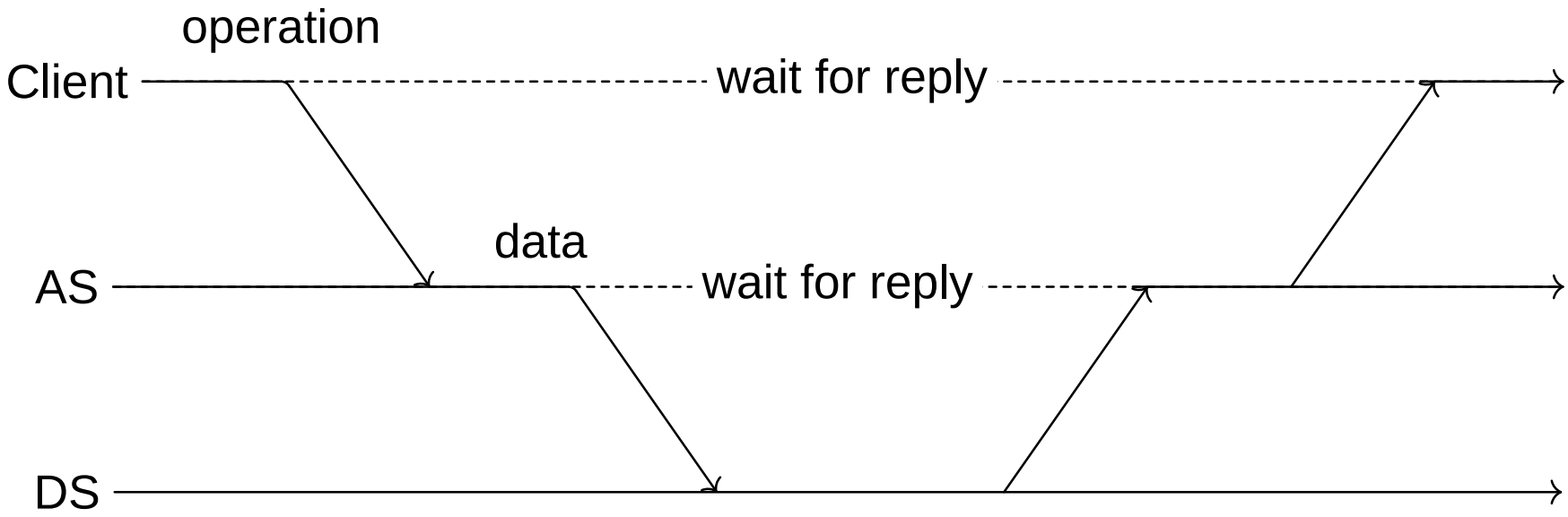
Multitiered Architectures: The 3-Tier Architecture

The most common multitiered organization is the 3-tier architecture, where the processing level is moved to its own dedicated server.

- **User-Interface Tier**: Resides on the client machine and handles interaction
- **Application Tier**: Resides on an intermediate server and contains the core business logic
  - e.g., application server
- **Data Tier**: Resides on a backend server responsible for persistent storage
  - e.g., database server

Multitiered Architectures: The 3-Tier Architecture

An example of an application server AS acting as client for a database server DS.



Multitiered Architectures: Vertical Distribution

Vertical distribution is achieved by placing logically different components on physically different machines.

- **Definition**: It follows the logical layering of the application, such as separating a Web server from a database server.
- **Functional Specialization**: This allows the hardware of the database server to be optimized for disk I/O, while the application server is optimized for high-performance computation.

Multitiered Architectures: Horizontal Distribution

Horizontal distribution involves physically splitting a single logical level into multiple, equivalent parts.

- **Scalability Mechanism**: A single logical server is replaced by a cluster of machines to balance the load.
- **Example - Server Clusters**: A Web-based service might use a front-end switch to distribute incoming requests across multiple replicated application servers.
- **Client-Side Horizontal Scaling**: Offloading simple processing (like form validation) to the client's machine is a form of horizontal distribution that reduces server load.

Case Study: Modern Search Engine

Tier	Functionality
User-Interface	Collects search descriptors and displays results.
Processing	Generates database queries and ranks retrieved records.
Data	Stores the massive collection of searchable records.

Feature	Vertical Distribution	Horizontal Distribution
Basis	Logical levels (UI, Logic, Data)	Replicas or partitions of one level
Scaling	Limited to adding tiers	Highly scalable by adding more machines
Goal	Modularization and management	Performance and load balancing

Symmetrically Distributed Architectures

Symmetrically Distributed Architectures

In **symmetrically distributed architectures**, the traditional distinction between a client and a server disappears. These systems are more commonly known as Peer-to-Peer (P2P) systems.

- **Role Equivalence**: Every process in the system has the same functional capabilities. A single process acts as both a client (requesting data) and a server (providing data) simultaneously.
- **Horizontal Distribution**: Symmetric architectures represent the logical extreme of horizontal distribution, where a single logical level is physically split into many equivalent parts across a large number of nodes.

Symmetrically Distributed Architectures

Decentralization and Autonomy

Symmetric architectures shift the focus from centralized control to decentralized coordination

- **No Central Authority**: There is no single point of control or failure. Nodes join and leave the network dynamically, a phenomenon often referred to as “churn.”
- **Resource Sharing**: Peers contribute resources (processing power, storage, or bandwidth) to the network, making the system’s total capacity grow as more participants join.
- **Direct Interaction**: Interactions occur directly between peers rather than through an intermediate server or broker.

Symmetrically Distributed Architectures

Client-Server vs. Symmetric

Feature	Client-Server	Symmetric (P2P)
Process Roles	Distinct (Client vs. Server).	Identical (Peers).
Coordination	Centralized.	Decentralized.
Scalability	Limited by server capacity.	Scales with the number of nodes.
Failure Impact	Server failure stops the system.	Individual node failure has minimal impact.

Symmetrically Distributed Architectures: Categories

While the underlying principle of symmetry is constant, the way peers are organized leads to different architectural styles:

- **Structured P2P**: The network topology is tightly controlled, and data is placed at specific locations
  - e.g., using Distributed Hash Tables
- **Unstructured P2P**: The network is formed randomly or ad-hoc, with no strict rules on where data is stored.
- **Hybrid P2P**: Combines elements of symmetry with some functional specialization, such as using “superpeers” to manage indices while ordinary peers store the data.

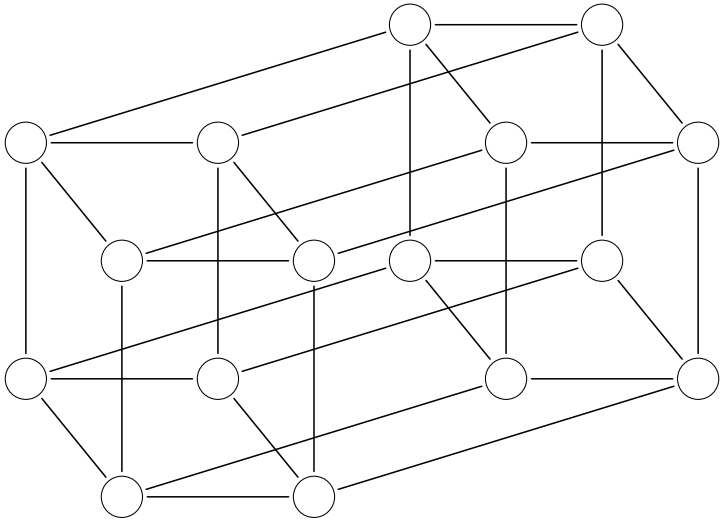
Structured Peer-to-Peer Systems

In a structured peer-to-peer (P2P) system, nodes (processes) are organized into an overlay network that follows a specific, deterministic topology.

- **Deterministic Topology**: Common structures include rings, binary trees, or grids.
- **Lookup Efficiency**: The primary goal of this strict organization is to ensure that data can be looked up efficiently.
- **Symmetric Roles**: Every process in the system typically carries out the same functional capabilities.

Structured Peer-to-Peer Systems: Organization Example

- Nodes are arranged in a binary hypercube structure
  - each node connected to others differing by one binary digit.
- Highly scalable and efficient for data-intensive applications.



Structured Peer-to-Peer Systems: Semantic-free Indexing

Structured systems are generally characterized by the use of a semantic-free index to manage and retrieve data.

- **Key Association**: Each data item maintained by the system is uniquely associated with a specific key.
- **Hashing Mechanism**: A hash function is used to generate these keys, typically defined as:

$$\text{key}(\text{data item}) = \text{hash}(\text{data item's value})$$

- **Independence**: The index is “semantic-free” because the key is derived from the data value rather than its meaning or metadata.

Structured Peer-to-Peer Systems: Distributed Hash Tables (DHT)

The system as a whole is responsible for storing pairs, effectively operating as a Distributed Hash Table (DHT).

- **Node Identifiers**: Each node in the system is assigned a unique identifier from the same set as the hash values used for data keys.
- **Data Responsibility**: Every node is made responsible for storing data associated with a specific, well-defined subset of the possible keys.
- **System View**: To the user, the DHT provides the same abstraction as a standard hash table, but with the storage distributed across a large-scale network.

Structured Peer-to-Peer Systems: Lookup Operation

The essence of a structured P2P system is its ability to map a key to the node responsible for it through a deterministic function.

- **Function Implementation**: The system provides an efficient implementation of a lookup function.
- **Mapping**: The function $\mathrm{lookup}(\mathrm{key})$ returns the network address of the existing node responsible for that specific key.
- **Deterministic Routing**: Because the topology is fixed, requests can be routed through the overlay with a guaranteed maximum number of hops.

Example: The Chord System

- **Ring Topology**: Nodes in the Chord system are logically organized into a ring structure using an m-bit identifier space.
- **Key Mapping**: A data item with an m-bit key k is assigned to the node with the smallest identifier $\mathrm{id} \geq k$
- **Successor Definition**: This responsible node is called the **successor** of key k, denoted as $\mathrm{succ}(k)$.

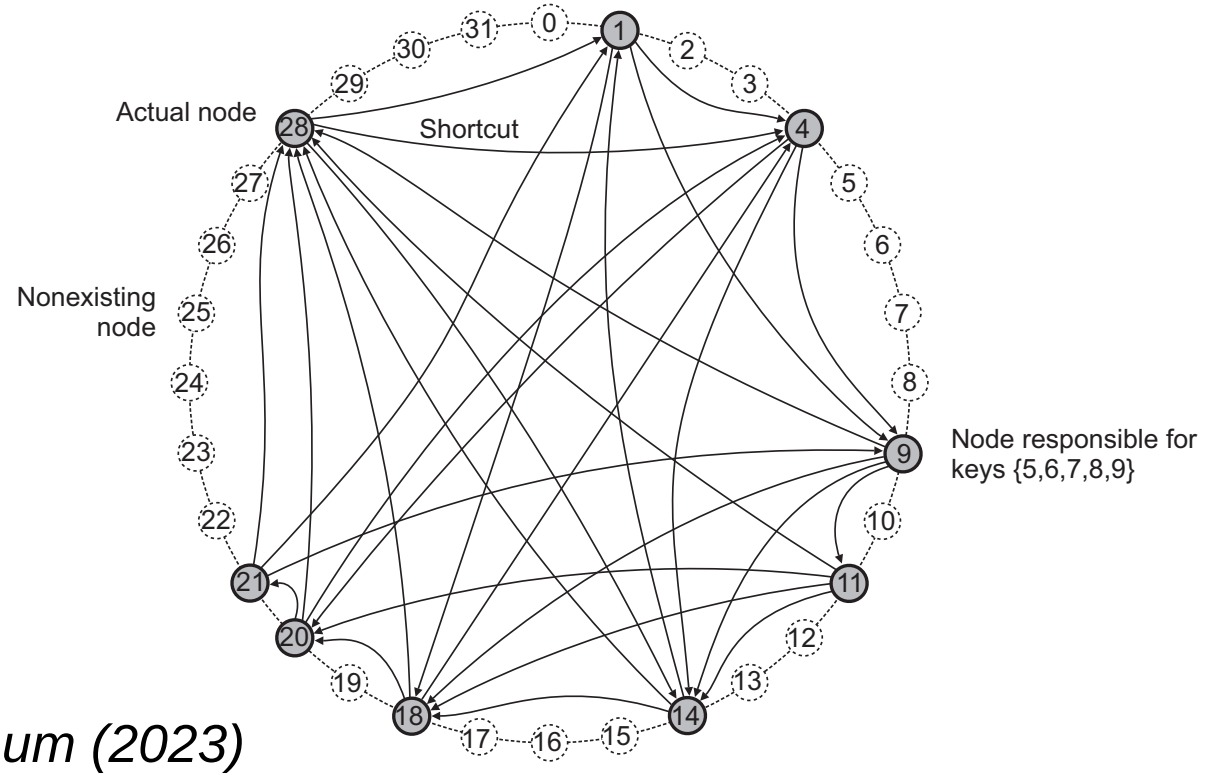
Efficient Lookups

- **Finger Tables**: Each node maintains a routing table, known as a finger table, which contains shortcuts to other nodes in the ring.
- **Expon. Shortcuts**: The i-th entry in a finger table at node p points to $\text{succ}(p + 2(i-1))$, allowing the distance to the target to decrease exponentially with each hop.
- **Performance**: This structured approach ensures that the number of hops required to resolve a key is $O(\log N)$, where N is the total number of nodes in the system.

Node Dynamics

- **Scalability**: Chord handles nodes joining and leaving by updating only a small number of finger table entries, ensuring the ring remains consistent.
- **Stabilization**: Nodes periodically run background procedures to verify their successors and predecessors, correcting any inconsistencies caused by concurrent joins or failures.

Example: The Chord System



Steen and Tanenbaum (2023)

Unstructured Peer-to-Peer Systems

The overlay network is organized in a non-deterministic or random fashion.

- **No Deterministic Mapping**: There is no fixed relationship between a data key and a specific node. A node does not know exactly where a specific piece of data is stored.
- **Ad-hoc Topology**: Each node maintains a list of neighbors, but these links are typically established through random processes or simple heuristic rules rather than a strict geometry like a ring.
- **Resilience to Churn**: Because the structure is loose, these systems are highly robust in environments where nodes frequently join and leave (high “churn”), as no complex global reorganization is required.

Unstructured Peer-to-Peer Systems: Search Mechanisms

Flooding

Since there is no index telling a node where data resides, the system must use search algorithms to locate items.

- **Mechanism**: A node looking for data sends a query to all its neighbors.
- **Propagation**: If a neighbor does not have the data, it forwards the query to all of its own neighbors.
- **Termination**: A **Time-To-Live (TTL)** value is attached to the query. Each hop decrements the TTL; when it reaches zero, the query is discarded.
- **Resource Usage**: Flooding is effective for finding popular items but generates significant network traffic.

Unstructured Peer-to-Peer Systems: Search Mechanisms

Random Walks

An alternative to flooding that aims to reduce network overhead is the random walk.

- **Mechanism**: A requesting node sends a query to one (or a few) randomly selected neighbor(s).
- **Process**: If that neighbor does not have the data, it forwards the request to one of its own random neighbors.
- **Trade-off**: Random walks produce much less network traffic than flooding but may take significantly longer to find the requested data and are less likely to find rare items.

Unstructured Peer-to-Peer Systems

Strengths and Weaknesses

Strengths	Challenges
Easy for nodes to join and leave without disrupting the system structure.	No guarantee that an existing item will be found (search is probabilistic).
Naturally supports complex queries, such as keyword searches or range queries.	High network overhead when using flooding in large-scale systems.
Highly robust against targeted attacks or massive node failures.	Inefficient for finding rare items that are only stored on a few nodes.

Hierarchically Organized Peer-to-Peer Networks

While pure peer-to-peer systems treat all nodes as equals, hierarchical architectures introduce a functional distinction to improve scalability and search efficiency.

- **Motivation**: In large-scale systems, the overhead of flat unstructured flooding or the complexity of structured routing can be mitigated by delegating coordination to more capable nodes.
- **Symmetry vs. Specialization**: These systems remain “peer-to-peer” because any node can potentially take on a higher-level role based on its resources and availability.

Hierarchically Organized Peer-to-Peer Networks

Superpeers and Weak Peers

The network is typically divided into two types of nodes with different responsibilities:

- **Superpeers**: Highly available nodes with significant bandwidth and processing power. They maintain an index of the data or services offered by their assigned weak peers.
- **Weak Peers**: Ordinary nodes that join and leave the network frequently. They do not participate in global coordination but rely on a superpeer to handle their search queries and presence.

Hierarchically Organized Peer-to-Peer Networks

Superpeer Organization

The superpeers themselves form an overlay network to ensure the system remains decentralized.

- **Structured Superpeer Overlay**: Superpeers can be organized into a DHT (like Chord). This allows for efficient, deterministic lookups across the global index.
- **Unstructured Superpeer Overlay**: Superpeers can be linked in a random graph. In this case, they use flooding or random walks among themselves to resolve queries that cannot be satisfied locally.

Hierarchically Organized Peer-to-Peer Networks

Trade-offs of Hierarchical P2P

Advantages	Challenges
Combines the efficiency of client-server (locally) with the scalability of P2P (globally).	**Superpeer Failure**: If a superpeer crashes, all its weak peers lose their connection to the network.
Reduces network traffic by limiting coordination to capable nodes.	**Complexity**: Requires robust algorithms to elect new superpeers and reassign weak peers during “churn”.

Hybrid System Architectures

Hybrid System Architectures

Real-world distributed systems are rarely built using a single, pure architectural style.

- **Complexity**: Modern systems combine various organizations
  - e.g., centralized features, peer-to-peer mechanisms, and hierarchical structures
- **Decentralization**: These architectures often cross organizational boundaries, leading to decentralized solutions where no single entity is responsible for the entire system
- **Goal**: The aim of hybrid architectures is to leverage the strengths of different models to meet modern application requirements.
  - e.g., the management of client-server and the scalability of P2P

Hybrid System Architectures: Major Categories

There are three primary types of hybrid architectures:

Architecture	Core Hybrid Nature
Cloud Computing	A highly advanced client-server model where the server is a massive, virtualized pool of distributed resources.
Edge-Cloud	Combines centralized cloud services with localized infrastructures “at the edge” to reduce latency and bandwidth.
Blockchain	Combines P2P decentralization with secure, immutable ledger structures to register transactions without a trusted third party.

Hybrid System Architectures: Rationale

Organizations adopt hybrid models to overcome the limitations of purely centralized or decentralized systems:

- **Scalability vs. Control**: Using cloud resources allows for dynamic scaling (pay-per-use) while maintaining a logically centralized management interface.
- **Performance vs. Reach**: Edge computing introduces an intermediate layer between end-devices (IoT) and the cloud to handle real-time processing that the cloud cannot support due to latency.
- **Trust vs. Transparency**: Blockchains allow untrusted participants to reach global consensus on transactions, physically replicating data across many nodes while maintaining a logically single chain.

Hybrid System Architectures: The Edge-Cloud Continuum

A significant trend in hybrid architecture is the “blurring” of boundaries between local and remote infrastructures.

- **Fog Computing**: Acts as the intermediate layer between the “edge” (close to devices) and the “cloud” (remote data centers).
- **Orchestration**: A key challenge in these hybrid systems is deciding which services to place on-premise for latency and which to move to the cloud for heavy computation.
- **Service Placement**: Hybrid designs must determine the optimal infrastructure for each service based on connectivity, hardware heterogeneity, and dynamic workloads.

Cloud Computing

It is s a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable resources (e.g., networks, servers, storage, applications, and services).

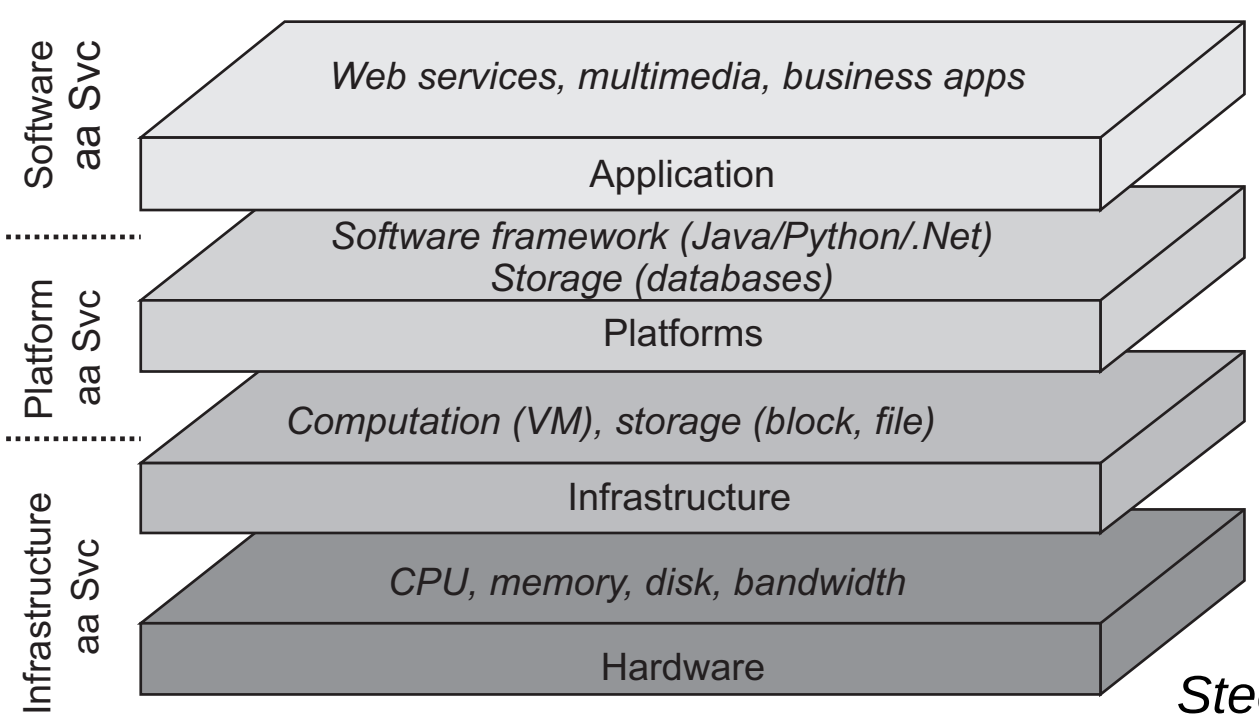
- **Logical Centralization**: From the user's perspective, the cloud acts as a single, powerful, and highly reliable server, even though it is physically implemented as a massive distributed system.
- **Evolution**: It represents an advanced evolution of the client-server model, where the "server" is no longer a single machine but a virtualized infrastructure.
- **Key Characteristic**: Resources can be rapidly provisioned and released with minimal management effort or service provider interaction.

Cloud Computing: Service Models (The Cloud Stack)

Cloud services are typically organized into three functional layers:

Model	Description
Infrastructure (IaaS)	Provides computing resources like virtual machines, storage, and networks. Users deploy their own operating systems and applications.
Platform (PaaS)	Provides a platform allowing customers to develop, run, and manage applications without the complexity of building the underlying infrastructure.
Software (SaaS)	Provides finished software applications over the internet (e.g., web-based email or office tools), managed entirely by the provider.

Cloud Computing: Service Models (The Cloud Stack)



Steen and Tanenbaum (2023)

Cloud Computing: Deployment Models

The architecture of a cloud depends on its accessibility and ownership:

- **Public Cloud**: The infrastructure is made available to the general public or a large industry group and is owned by an organization selling cloud services.
- **Private Cloud**: The infrastructure is operated solely for a single organization. It may be managed by the organization or a third party.
- **Community Cloud**: Shared by several organizations and supports a specific community that has shared concerns (e.g., mission, security requirements, or policy).
- **Hybrid Cloud**: A composition of two or more clouds (private, community, or public) that remain unique entities but are bound together by standardized technology.

Cloud Computing: Virtualization (The Enabler)

Virtualization is the core mechanism that allows cloud providers to manage physical resources efficiently.

- **Resource Pooling**: Providers serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned.
- **Independence**: Virtual machines (VMs) provide isolation, ensuring that the activities of one customer do not affect the security or performance of another on the same physical hardware.
- **Flexibility**: Virtualization allows for the seamless migration of services between physical nodes to balance load or perform maintenance without downtime.

Cloud Computing: Elasticity and Scaling

One of the most significant advantages of cloud architecture is its ability to handle dynamic workloads.

- **Rapid Elasticity**: Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand
- **Measured Service**: Cloud systems automatically control and optimize resource use by leveraging a metering capability
  - e.g., storage, processing, bandwidth, and active user accounts
- **Cost Efficiency**: Users typically follow a “pay-per-use” model, eliminating the need for large upfront capital investments in hardware

Edge-Cloud Architecture

The proliferation of the Internet of Things (IoT) has led to a massive increase in data generated at the boundary of the network.

- **Cloud Limitations**: While cloud computing offers massive resources, the latency and bandwidth requirements for processing real-time IoT data make a purely centralized approach infeasible.
- **Hybrid Approach**: The edge-cloud architecture combines localized processing near the data source with the extensive resources of remote data centers.

Edge-Cloud Architecture: Three-Layered Organization

Modern edge-cloud systems are typically organized into a three-layered hierarchy to balance processing needs and network constraints:

Layer	Role and Characteristics
Edge Layer	Contains the actual devices (sensors, actuators) that produce data or interact with the physical world.
Fog Layer	An intermediate layer of localized infrastructure that provides low-latency processing and storage.
Cloud Layer	Remote, centralized data centers providing high-performance computing and massive long-term storage.

Edge-Cloud Architecture: Edge Layer

The edge layer represents the physical interface of the distributed system.

- **Devices**: Includes smartphones, smart sensors, cameras, and wearable devices
- **Constraints**: These devices are often limited in terms of battery life, processing power, and memory
- **Function**: They are responsible for data acquisition and simple local actions
  - e.g., a smart thermostat turning off a heater

Edge-Cloud Architecture: Fog Layer

Fog computing acts as the bridge between the edge and the cloud, situated physically close to the end-devices.

- **Localized Infrastructure**: Consists of local gateways, routers, or specialized small-scale servers (cloudlets).
- **Low Latency**: By processing data locally, the fog layer provides the near-instantaneous response times required for applications like autonomous driving or industrial automation.
- **Bandwidth Optimization**: It filters and aggregates data before sending only the most relevant information to the cloud, significantly reducing wide-area network traffic.

Edge-Cloud Architecture: Cloud Layer

The cloud layer remains the backbone for deep analysis and global management.

- **Global Analytics**: Used for processing historical data from thousands of edge locations to identify long-term trends or train machine learning models
- **Edge-Cloud Continuum**: There is no rigid boundary; instead, services are placed dynamically along the “continuum” based on current network conditions and application needs
- **Orchestration**: A critical challenge is determining which specific sub-tasks of a service should run in the cloud versus the fog

Edge-Cloud Architecture: Advantages

- **Improved Responsiveness**: Drastically reduces round-trip times by avoiding long-distance communication to remote data centers
- **Enhanced Privacy**: Sensitive data can be processed and stripped of personally identifiable information (PII) at the fog layer before being transmitted elsewhere
- **Reliability**: Systems can continue to function locally (at the edge/fog) even if the connection to the main cloud provider is temporarily lost

Blockchain Architectures

A blockchain is a specific type of Distributed Ledger Technology (DLT) that maintains a continuously growing list of records, called blocks, which are secured using cryptography.

- **Logically Centralized, Physically Distributed**: While the architecture is physically a decentralized peer-to-peer network, it maintains a logically single, consistent ledger of transactions
- **Hybrid Nature**: It combines elements of symmetric (P2P) architectures for communication with structured data management to ensure a global “truth”
- **The Ledger**: Acts as a public or private database where every transaction is recorded and cannot be altered retrospectively without changing all subsequent blocks

Blockchain Architectures: Structural Organization

The architecture is defined by how data is grouped and linked over time:

- **Blocks**: Transactions are bundled together into a block
- **Chaining**: Each block contains a cryptographic hash of the previous block, creating a link that traces back to the very first block (the genesis block)
- **Immutability**: Because each block is linked via hashes, any attempt to change a past transaction would invalidate the hashes of every subsequent block in the chain

Blockchain Architectures: Peer-to-Peer Foundation

Blockchain architectures rely on a symmetric distribution model to ensure resilience and transparency:

- **Node Roles**: Every node in the network maintains a complete copy of the entire blockchain
- **Transaction Propagation**: When a new transaction is initiated, it is broadcast to all peers in the network
- **Verification**: Nodes independently verify the validity of transactions before they are included in a block
- **No Central Authority**: Decisions regarding the state of the ledger are made through distributed coordination rather than a central server

Blockchain Architectures: Permissionless vs. Permissioned

Blockchain architectures are categorized based on who is allowed to participate in the network:

Feature	Permissionless (Public)	Permissioned (Private)
Participation	Open to anyone; anyone can read, write, or audit.	Restricted to a specific group of known, pre-verified participants.
Identity	Users are often pseudonymous.	Participants have established identities.
Consensus	Requires heavy mechanisms like Proof of Work (PoW).	Uses lighter, faster protocols among trusted entities.
Control	Fully decentralized.	Consortium or organization controlled.

Blockchain Architectures: Advantages and Challenges

- **Transparency**: Every participant can audit the entire history of the system.
- **Security**: The decentralized nature and cryptographic linking make the system highly resistant to tampering and single points of failure
- **Challenges**:
  - **Scalability**: Replicating the entire ledger on every node leads to high storage and bandwidth requirements
  - **Latency**: Reaching global consensus across a P2P network is significantly slower than centralized processing

The End

Resources

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.